



CineMatch

A Content-Based Movie Recommendation System

Final Project Report

Submitted by: Yug Adhikari

Student ID: 25123860

Course / Program: BSc. (Hons) Computer Science with AI

Submitted to: Umesh Regmi

Date: 2nd Oct 2026

Abstract

The project is called CineMatch, it is a content-based movie recommendation system which recommends movies similar to a movie selected by the users. Instead of collaborative-filtering systems, which use ratings from other users, CineMatch uses information from each movie's own metadata: its keywords, genres, principal cast, director, and plot overview and recommends movies that have the closest thematic profile as computed based on cosine similarity for a bag-of-words representation of the plot.

The system is provided as a full stack Django web app with a machine learning model trained and tested in a Jupyter notebook using pandas and scikit-learn. The trained model artifacts are loaded by the Django backend to provide recommendations on the fly. This application also showcases an external API integration (The Movie Database API for show posters, it's optional) as well as database integration (logging the search history using the Django ORM).

Table of Contents

1. Introduction
2. Objectives
3. Literature Review
4. System Requirements
5. System Design and Architecture
6. Dataset Description
7. Machine Learning Methodology
8. Django Web Application Implementation
9. Database Integration
10. API Integration
11. Results and Screenshots
12. Testing
13. Limitations and Future Work
14. Conclusion
15. References

1. Introduction

One of the most apparent uses of machine learning in real-world situations is recommendation systems, which drive movie, product, and content recommendations shown to users on platforms like Netflix, Amazon, and YouTube. Generally, these systems can be divided into two categories: collaborative filtering systems make recommendations based on patterns observed over a large number of users' activities and content-based filtering systems make recommendations based on the characteristics of the specific items that have already been of interest to the user.

This project is an implementation of the content based movie recommendation system. When the user gives one movie name that he/she likes, the system spits out a list of the most similar movies (just according to the genre, cast, director, keywords, and plot summary of the movie). This was selected because of the lack of need to use ratings history of users (and thus avoiding the “cold start” problem of collaborative filtering) and because it is appropriate for the scope of this project – the entire pipeline, from the raw data to a working web interface, can be understood, implemented, and explained by one developer.

After completing the project, a user types or selects a movie by name and is given a list of recommended movies, each with a poster, genres, rating and a short summary. The final product is a Django web app called CineMatch, where a user can enter or select a movie and be shown a list of movies that they are recommended. Each movie is displayed with its poster, genres, rating, and a brief description.

2. Objectives

- To create a real-world movie metadata based content based recommendation model.
- Reduces the problem to a form of similarity comparison between text.
- To train and test the model using a well documented Jupyter notebook.
- To incorporate the trained model into a Django Web Application using a clean and usable interface.
- To show how to integrate search history into the database.
- Show how to optionally pull in real movie posters to show external API integration.
- To prepare the project for third party understanding, running and evaluation.

3. Literature Review

Typically, there are three approaches to recommendation systems research. The collaborative filtering approach (as in Netflix, Amazon) predicts a user's interest in an item, based on the preferences of similar users, but requires a great deal of historical rating data to be effective. In this project, the similarity between items is measured directly using the attributes that describe them (content filtering); this is the approach taken by many news and article recommender systems, as well as some movie recommender systems prototypes developed with the TMDb and MovieLens datasets. Both have been used but hybrid systems are used to balance out the weaknesses of either.

In the case of text-similarity tasks like this one, two typical vectorization methods will be used: a simple Bag-of-Words / Count Vectorizer, and TF-IDF (Term Frequency–Inverse Document Frequency), which reduces the impact of words that appear in a high number of documents. CountVectorizer is employed with cosine similarity, a common and well-documented methodology for small-to-medium sized content-based recommenders, which is used in this project for its simplicity and interpretability.

4. System Requirements

4.1 Software Requirements

Component	Technology
Programming Language	Python 3.10+
Web Framework	Django 5.0
Machine Learning	scikit-learn, pandas, numpy
Database	SQLite (default Django database)
Frontend	HTML5, CSS3
Notebook Environment	Jupyter Notebook
External API (optional)	TMDB (The Movie Database) API

4.2 Hardware Requirements

- A standard laptop/desktop with at least 4GB RAM
- Internet connection (for installing dependencies and, optionally, fetching posters)

5. System Design and Architecture

The system is organized into two main stages: an offline training stage (the Jupyter notebook) and an online serving stage (the Django web application).

5.1 High-Level Architecture

Offline (once): Raw dataset (CSV) → Data cleaning & feature engineering → CountVectorizer → Cosine similarity matrix → Saved artifacts (movies.pkl, similarity.pkl)

Online (every request): User selects a movie on the web page → Django view loads the saved artifacts → Looks up the movie's similarity row → Sorts and returns the top-N most similar movies → Poster is fetched from TMDB (optional) → Search is logged to the database → Results rendered back to the user

5.2 Django App Structure

File	Responsibility
recommender/models.py	Defines SearchHistory (database table)
recommender/recommender_engine.py	Loads movies.pkl / similarity.pkl and computes recommendations
recommender/poster_utils.py	Fetches poster images from the TMDB API, with fallback
recommender/views.py	index (search + results), history (past searches), api_recommend (JSON API)
recommender/urls.py	Routes for the app
recommender/templates/	index.html, history.html, base.html
recommender/static/	style.css, placeholder poster image

6. Dataset Description

The project uses a merged version of the TMDb 5000 Movie Dataset, a well-known public dataset originally sourced from The Movie Database (TMDB) and widely used for recommendation-system projects. It contains 4,803 movies with the following relevant fields:

- title — the movie's title
- genres — one or more genres (e.g. Action, Drama, Comedy)
- keywords — plot-related tags
- cast — principal actors
- director — the movie's director
- overview — a short plot synopsis
- vote_average, release_date — used for display purposes

The dataset is provided as `movie_dataset.csv` in the `dataset/` folder of this submission.

7. Machine Learning Methodology

Notebook `Movie_Recommendation_Model.ipynb` contains full implementation detail, code and outputs. The steps are summarized below.

7.1 Data Cleaning

Rows with missing genres, keywords, cast, director, or overview had those fields replaced with empty strings so they would not break the text-combination step, rather than dropping rows and losing data.

7.2 Feature Engineering

For each movie, the genres, keywords, cast, director, and overview fields were concatenated into a single text field called "tags", lower-cased for consistency. This tag string represents the movie's full content profile in one place.

7.3 Vectorization

The tags column was converted into numeric vectors using scikit-learn's `CountVectorizer`, configured to keep the 5,000 most frequent terms and to remove common English stop words. This produced a $4,803 \times 5,000$ matrix, where each row is a movie and each column represents how often a given word appears in that movie's tags.

7.4 Similarity Computation

Cosine similarity was computed between every pair of movie vectors, producing a $4,803 \times 4,803$ similarity matrix. Cosine similarity measures the angle between two vectors rather than their magnitude, which suits text data well since it is not affected by differences in text length. The matrix was cast to `float32` to reduce the saved file size.

7.5 Recommendation Function

Given a movie title, the corresponding row of the similarity matrix is retrieved, sorted in descending order, and the top N (excluding the movie itself) are returned as the recommended movies.

7.6 Evaluation

Because this is an unsupervised, content-based system with no ground-truth ratings, it was evaluated qualitatively (by manually inspecting recommendations for well-known movies such as Avatar and The Dark Knight Rises) and with a simple genre-overlap metric: for a sample of query movies, the fraction of their top-5 recommendations that share at least one genre with the query. This offers a rough, automatic sanity check that the recommendations are thematically consistent; the full calculation and results are in the notebook.

7.7 Saving the Model

The cleaned movies table and the similarity matrix were serialized with Python's pickle module as movies.pkl and similarity.pkl. These two files are the only artifacts the Django application needs — it never re-runs the training pipeline at request time.

8. Django Web Application Implementation

The web application is a single Django app named recommender inside the project movie_recommender_django.

8.1 Loading the Model

recommender_engine.py loads movies.pkl and similarity.pkl once, the first time it is imported (i.e. when the Django server starts), and keeps them in memory. This means every subsequent web request is served from memory instantly, with no disk I/O or retraining.

8.2 Views

- index — renders the search form; if a movie was submitted, computes recommendations, attaches a poster URL to each, logs the search, and re-renders the page with results.
- history — displays the 20 most recent searches, read from the database.
- api_recommend — a JSON endpoint at /api/recommend/<movie_title>/ for programmatic access, returning the same recommendation data as JSON.

8.3 Templates and Styling

Three templates (base.html, index.html, history.html) share a common layout. A custom stylesheet (style.css) gives the app a distinctive dark, cinema-marquee visual theme rather than default Django/Bootstrap styling.

9. Database Integration

The project uses Django's built-in ORM with the default SQLite database. A single model, SearchHistory, stores every movie a user searches for along with a timestamp:

```
class SearchHistory(models.Model):
    movie_title = models.CharField(max_length=255)
    searched_at = models.DateTimeField(auto_now_add=True)
```

Every successful search on the home page creates a new SearchHistory row. The /history/ page then queries and displays the most recent 20 entries, demonstrating a full read/write cycle through the Django ORM. The model is also registered with the Django admin site for inspection.

10. API Integration

To make the results visually complete, each recommended movie can optionally display its real poster image, fetched from The Movie Database (TMDB) public API. `poster_utils.py` sends a search request to TMDB's `/search/movie` endpoint using the movie title and reads the `poster_path` from the first result.

If no `TMDB_API_KEY` environment variable is configured, or the request fails for any reason (no internet access, rate limiting, movie not found on TMDB), the function safely falls back to a local placeholder image rather than breaking the page. This makes the feature fully optional: the application works correctly with or without an API key.

11. Results and Screenshots

Below is a sample output from the training notebook showing the distribution of the most common genres in the dataset:

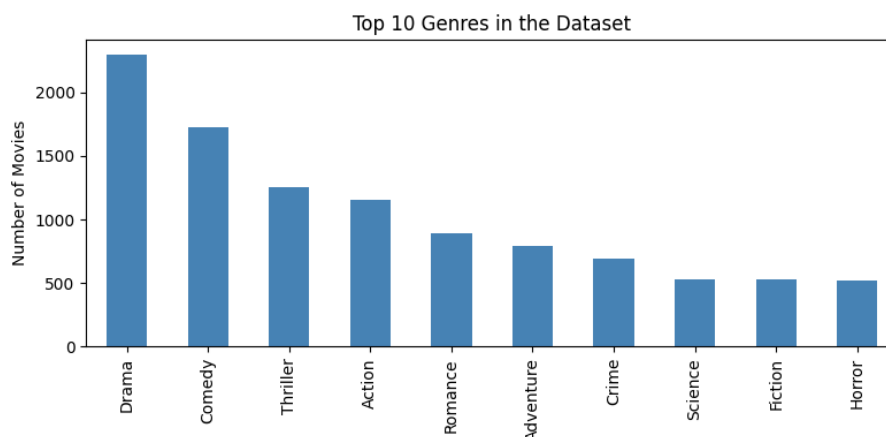
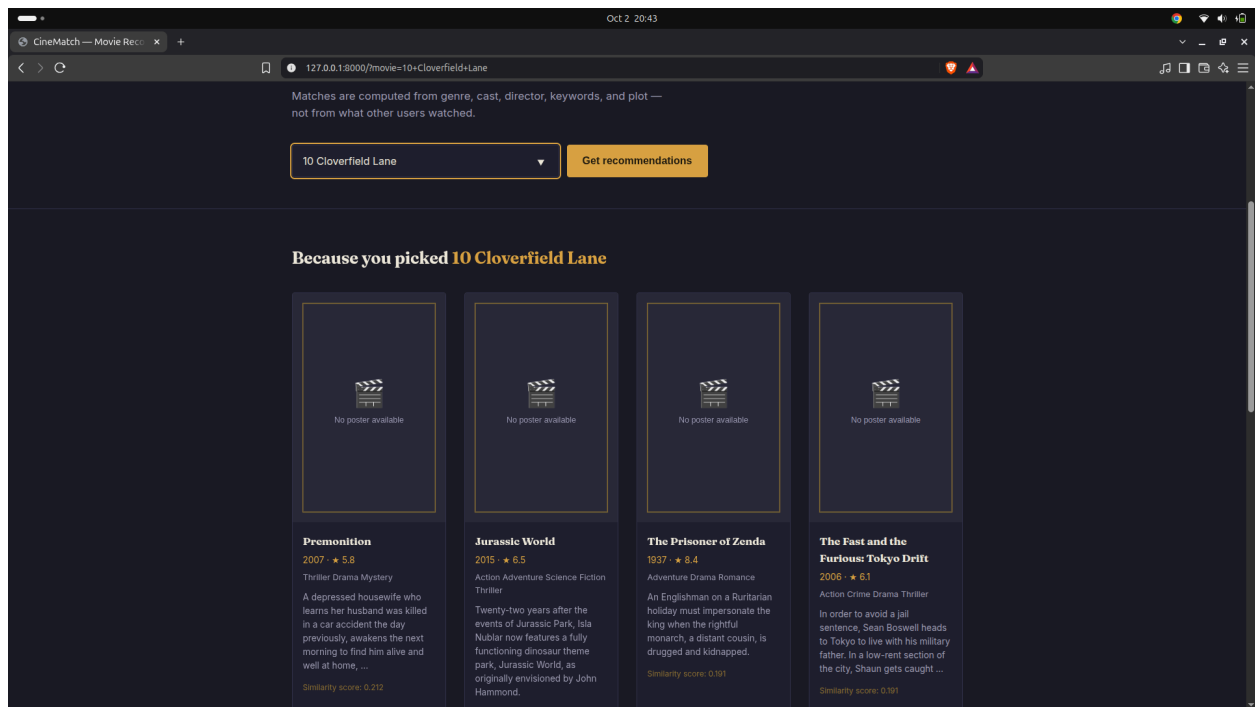
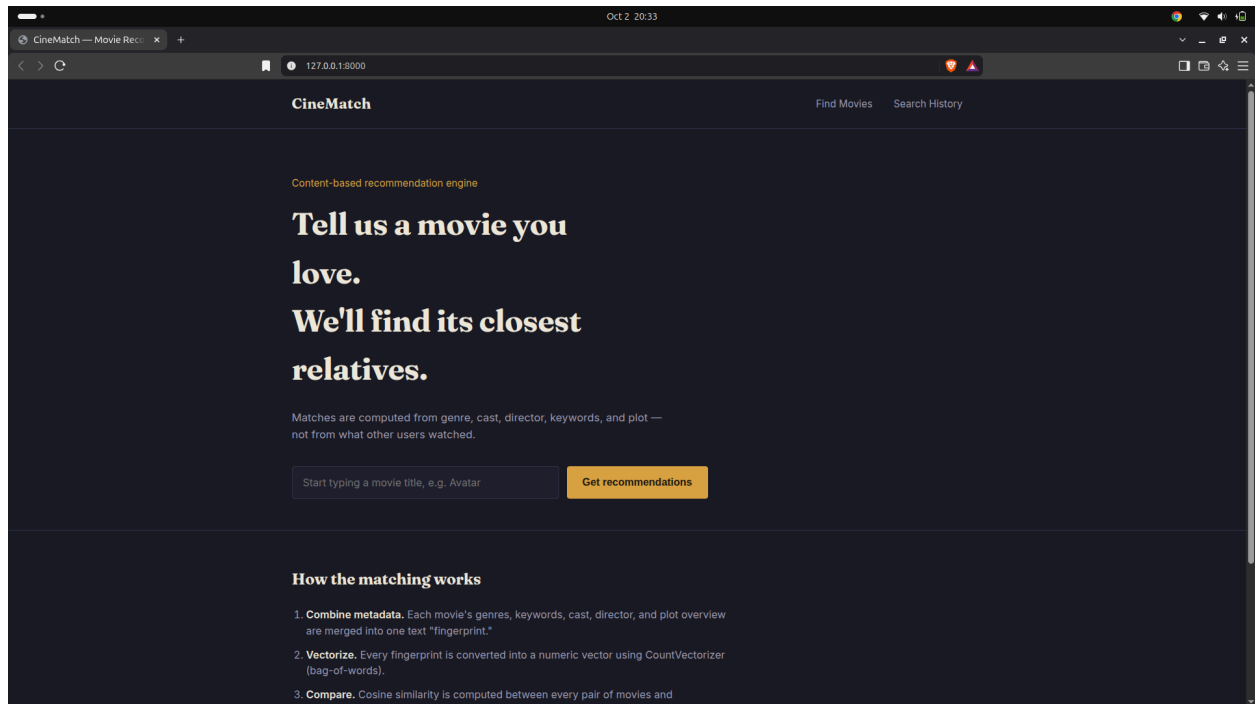
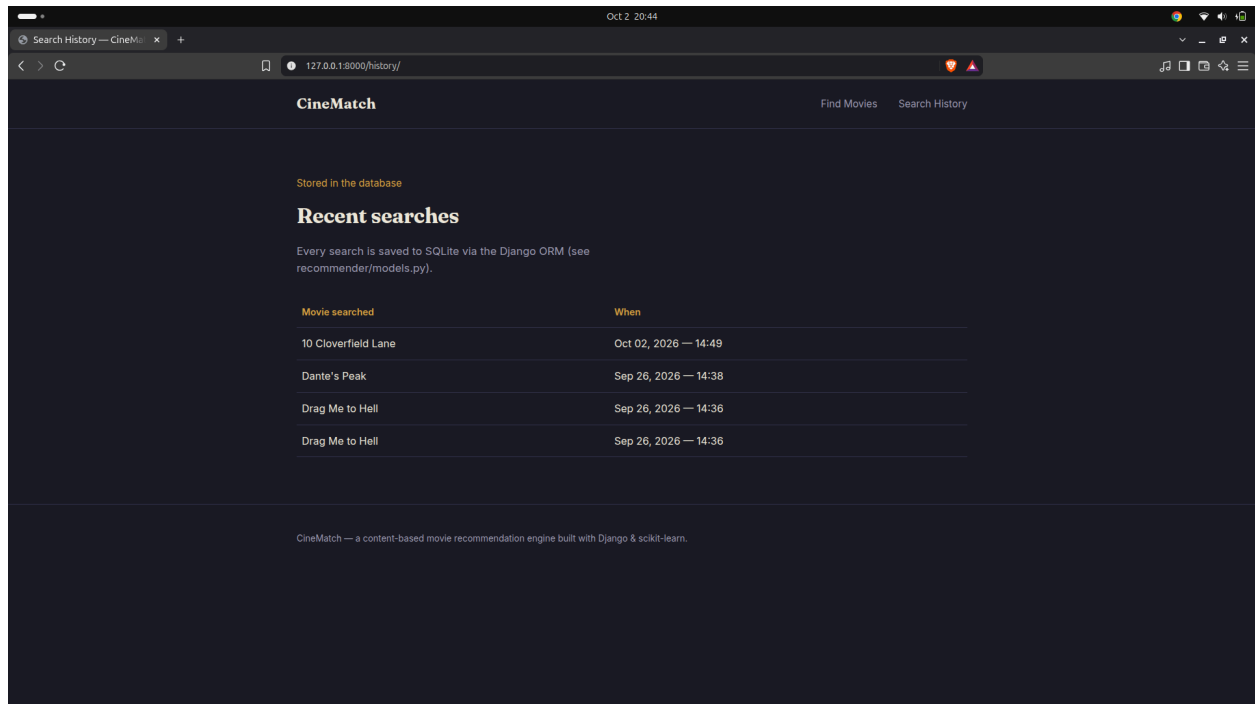


Figure 1: Top 10 genres in the training dataset (generated in the notebook).

11.1 Application Screenshots





12. Testing

The application was manually tested end-to-end before submission:

Test Case	Steps	Result
Home page loads	GET /	200 OK — search form renders
Valid search	Search for "Avatar"	200 OK — 6 relevant recommendations returned
Invalid search	Search for a title not in the dataset	Friendly error message shown, no server error
JSON API — valid	GET /api/recommend/Avatar/	200 OK — JSON list of recommendations
JSON API — invalid	GET /api/recommend/NotAMovie/	404 with a JSON error message
Database logging	Search, then open /history/	Search appears in the history table
Django system check	python manage.py check	No issues identified

13. Limitations and Future Work

- Content-based filtering only considers a movie's own attributes; it does not learn from what other users actually liked, so it may miss non-obvious connections that collaborative filtering could find.
- The search requires an exact (case-insensitive) title match; a fuzzy-search/autocomplete-only interface would be more forgiving of typos.
- The similarity matrix is precomputed and does not automatically update if new movies are added — the notebook would need to be re-run.
- Future work could add collaborative filtering (using user ratings) for a hybrid recommender, user accounts to personalize history, and a larger, more recent movie dataset.

14. Conclusion

This project successfully implements a complete, working content-based movie recommendation system, from raw data to a deployed web interface. It demonstrates the full machine learning workflow (data cleaning, feature engineering, vectorization, similarity computation, and evaluation) as well as practical software engineering skills: structuring a Django project, integrating a trained ML model into a web backend, persisting data with an ORM, and consuming an external API safely. The result, CineMatch, is a usable application that gives sensible, explainable movie recommendations in real time.

15. References

1. The Movie Database (TMDB) — TMDB 5000 Movie Dataset. <https://www.themoviedb.org/>
2. Django Software Foundation — Django Documentation. <https://docs.djangoproject.com/>
3. Pedregosa, F. et al. — Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 2011. <https://scikit-learn.org/>
4. McKinney, W. — pandas: a Foundational Python Library for Data Analysis. <https://pandas.pydata.org/>
5. The Movie Database API Documentation. <https://developer.themoviedb.org/>